

Distributed Multi-Model Analytics for E-commerce Data

Name: Umuhoza Fabiola

Student ID: 101361

Course: Big Data Analytics

Table of Contents

Abstract 4

1. Introduction and business objective 5

1.1 Background . . . 5

1.2 Business objective . . . 5

1.3 Analytical questions . . . 5

1.4 Objectives of the implementation . . . 5

1.5 Scope and honesty about evidence . . . 6

1.6 Structure of the report . . . 6

2. Architecture 6

2.1 Component responsibilities . . . 7

2.2 Design principles . . . 7

3. Data description 7

3.1 Source and sample size . . . 7

3.2 Entities and key fields . . . 8

3.3 Event types, referrers and devices . . . 8

3.4 Data preparation rules . . . 8

3.5 Data quality observations . . . 9

4. Data modelling and technology selection 10

4.1 MongoDB . . . 10

4.2 HBase . . . 12

4.3 Choosing between MongoDB and HBase . . . 15

5. Spark batch processing and SQL 15

5.1 Pipeline overview . . . 15

5.2 Category revenue . . . 15

5.3 Product pairs (basket analysis) . . . 16

5.4 Cohort revenue . . . 16

5.5 Funnel query in Spark SQL . . . 16

5.6 Scale-oriented implementation choices . . . 16

5.7 Spark evidence . . . 16

6. Integrated analysis: acquisition-to-purchase funnel **17**

6.1 Why a cross-system join is needed . . . 17

6.2 Measured finding . . . 18

6.3 Statistical check added to this report . . . 18

6.4 Decision implications . . . 19

7. Results and visualisations **20**

7.1 Category performance (Q1) . . . 20

7.2 Frequently bought together (Q2) . . . 21

7.3 Cohort revenue (Q3) . . . 22

7.4 Acquisition funnel (Q4) . . . 24

7.5 Interactive dashboard . . . 25

8. Discussion **26**

8.1 Answers to the analytical questions . . . 26

8.2 Business interpretation . . . 26

8.3 Validity and threats . . . 27

8.4 Performance and scalability . . . 27

8.5 Design trade-offs revisited . . . 27

9. Limitations and future work **28**

9.1 Limitations . . . 28

9.2 Future work . . . 28

10. Conclusion **28**

11. Reproducibility **29**

11.1 Suggested run order . . . 29

References **29**

Appendix A. Data dictionary **30**

Appendix B. Evidence checklist **31**

Appendix C. Environment and versions **32**

Appendix D. Glossary **32**

Abstract

This report describes the design, implementation and evaluation of a distributed, multi-model analytics pipeline for e-commerce data. The work uses a synthetic 90-day data set of 500 users, 300 products, 2,000 transactions and 10,000 browsing sessions. Business documents (categories, products, users and transactions with embedded line items) are modelled in MongoDB, while high-volume, sparse page-view events and derived product-day metrics are modelled in Apache HBase. Apache Spark combines both stores, normalises embedded line items, and runs batch and Spark SQL analyses.

Four business questions are answered: which product categories drive revenue, which products are bought together, how customer cohorts defined by registration month spend over time, and which acquisition channel converts browsing sessions into purchases. In the 10,000-session sample, gross line revenue was $1,895,377 and 1,001 sessions were linked to a purchase, an overall session conversion rate of 10.01%. Category cat_019 led revenue at $126,678. Email had the highest channel conversion at 10.76% and search-engine traffic the lowest at 9.15%.

A supplementary statistical check added in this version of the report shows that the differences between channels are not statistically significant at the 5% level (chi-square test across five channels, $p = 0.52$; email versus search engine, two-sided $p = 0.088$). The channel result should therefore be treated as a hypothesis to monitor rather than as grounds for a large budget shift. All results are descriptive and come from generated data, not production customer or marketing data.

Keywords: big data analytics, MongoDB, HBase, Apache Spark, polyglot persistence, funnel analysis, cohort analysis, market-basket analysis.

1. Introduction and business objective

1.1 Background

Online retailers generate two very different kinds of data. The first is transactional and descriptive: catalogue entries, customer profiles, and orders whose line items are naturally read together as one business object. The second is behavioural: a continuous stream of page views, searches, cart actions and checkouts, produced in far higher volume and mostly read back by time range or by customer. A single storage engine is rarely the best fit for both. Large analytical questions then require the two kinds of data to be brought back together, which is the role of a distributed processing engine.

This project follows that pattern, often called polyglot persistence. Each workload is stored in the system whose data model and access paths suit it best, and Apache Spark is used for the joins and aggregations that span systems. The project analyses a synthetic 90-day e-commerce data set so that the full pipeline can be built, run and inspected on a single machine while keeping the same design that would be used at larger scale.

1.2 Business objective

The objective is to identify revenue-driving product categories, product affinities, customer cohorts, and acquisition channels that convert browsing into purchases. The results are intended to support decisions about assortment, cross-selling, retention and marketing spend.

1.3 Analytical questions

- **Q1 Category performance.** Which product categories generate the most revenue and units?
- **Q2 Product affinity.** Which pairs of distinct products are most often bought in the same order, and are they strong enough to drive "frequently bought together" suggestions?
- **Q3 Cohorts.** How does revenue vary by the month customers registered and the month they purchased?
- **Q4 Acquisition funnel.** Which referrer produces the greatest share of converting sessions, and how should marketing budget be prioritised?

1.4 Objectives of the implementation

- Design a MongoDB document model and indexes for catalogue, user and order data, with embedded line items.
- Design HBase row keys and column families for user event retrieval and product-day metrics.
- Use Spark to read all JSON partitions, normalise embedded items, join the two systems and write compact result tables.
- Present the results as static charts and an interactive dashboard, and document the evidence that each component actually ran.

1.5 Scope and honesty about evidence

The report distinguishes between what was executed and evidenced, and what is designed but not yet evidenced. Spark outputs, the dashboard, the HBase deployment and an HBase prefix scan are evidenced by screenshots included in this report. MongoDB Compass screenshots show that the collections are loaded, but the MongoDB aggregation outputs themselves have not yet been captured, so no aggregation results are claimed (see Section 4.1 and Appendix B).

1.6 Structure of the report

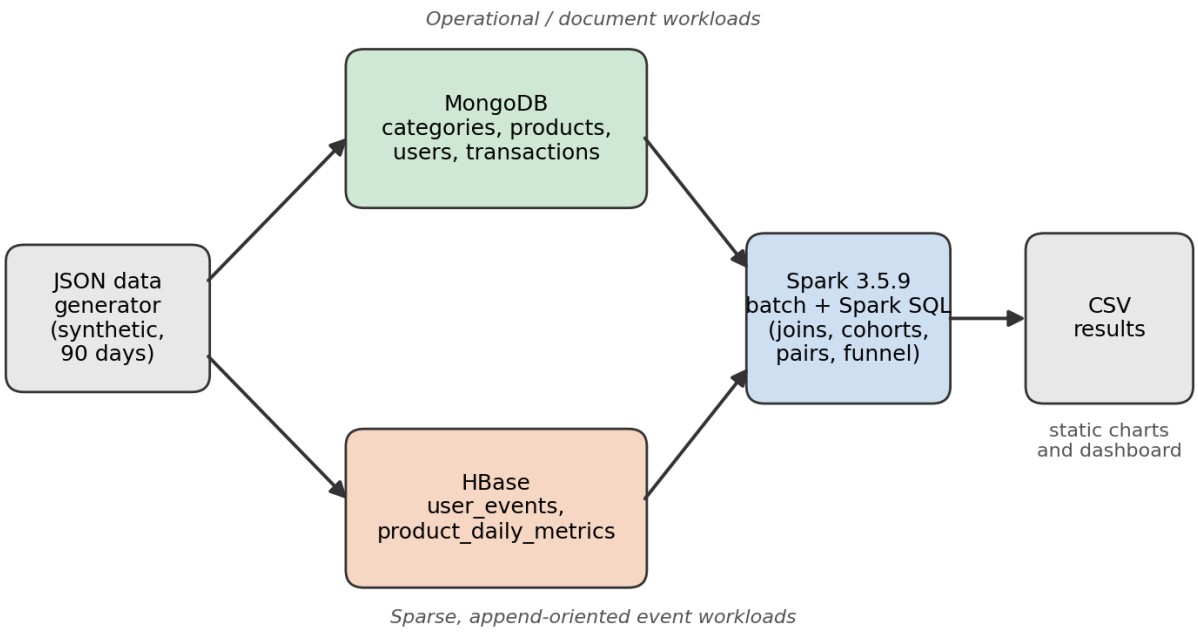
Section 2 presents the architecture. Section 3 describes the data. Section 4 covers data modelling and technology selection for MongoDB and HBase. Section 5 describes the Spark processing. Section 6 presents the integrated funnel analysis. Section 7 reports the results for all four questions. Section 8 discusses the findings, validity and scalability. Section 9 lists limitations and future work, Section 10 concludes, and Section 11 covers reproducibility. Appendices give the environment, a data dictionary, an evidence checklist and a glossary.

2. Architecture

The architecture separates operational and document workloads from high-volume event retrieval and uses Spark for joins and batch analytics. Data flows from a JSON generator into two stores, from both stores into Spark, and from Spark into CSV result files that feed static charts and the dashboard.

```
JSON generator -> MongoDB: users, products, transactions
                -> HBase: flattened page-view events, product-day metrics
MongoDB + HBase -> Spark batch / Spark SQL -> CSV results -> static charts
```

Figure 1. Architecture and data flow of the pipeline



2.1 Component responsibilities

Component	Holds / does	Why it is used
JSON generator	Produces users, products, transactions and sessions as JSON partitions	Repeatable synthetic data with no privacy risk
MongoDB	Complete business documents: categories with embedded subcategories, products with price history, users with nested geography, transactions with embedded items	Nested documents are read as one object; flexible aggregation framework
HBase	Sparse, append-oriented browsing events and derived product-day metrics	Row-key range scans retrieve one user or product over time without scanning others
Spark 3.5.9	Reads all JSON partitions, explodes embedded items, joins sessions to transactions, computes pairs, cohorts and the funnel	Parallel joins and aggregations that cannot be done efficiently by cross-system nested loops
CSV, charts, dashboard	Compact aggregate tables and their visualisations	Portable, inspectable results

2.2 Design principles

- **Query-first modelling.** Each store is modelled around the reads it must serve, not around a normalised schema.
- **Deliberate duplication.** Small derived metrics are duplicated in HBase because HBase does not support joins and must answer from a single row-key range.
- **Move computation to the data.** Cross-system questions are answered by a Spark join that returns a compact table, rather than by application loops that fetch rows one at a time.
- **Reduce data early.** Only required columns are selected before joins, and only small aggregates are collected on the driver.
- **Evidence before claims.** A result is reported as executed only when there is output or a screenshot to support it.

3. Data description

3.1 Source and sample size

The data are generated by a script and are synthetic. The local sample used for all results contained 500 users, 300 products, 2,000 transactions and 10,000 sessions, covering a 90-day activity window. Spark produced 25 category rows, 2,682 product pairs, 27 cohort-month combinations and 5 referrer rows. Generated data is excluded from version control, so the results can be reproduced by re-running the generator rather than by committing data files.

Entity	Records in sample	Primary store	Note
Categories	25	MongoDB	Each with an embedded subcategory list

Entity	Records in sample	Primary store	Note
Products	300	MongoDB (+ derived metrics in HBase)	Embedded price_history array
Users	500	MongoDB	Nested geographic data
Transactions	2,000	MongoDB	Embedded items array
Sessions	10,000	HBase events (read by Spark from JSON)	Referrer and device context

3.2 Entities and key fields

Appendix A contains a field-level data dictionary. In summary, a **category** carries an identifier, a name and an embedded subcategory list. A **product** carries a product identifier used as its document key, a name, a category identifier, a base price, current stock, an active flag, an embedded price history and a creation date. A **user** carries a user identifier, nested geographic data, a registration timestamp and a last-active timestamp. A **transaction** carries a transaction identifier, user and session identifiers, order totals and status, and an embedded array of line items. A browsing **event** carries a user, a timestamp, a session identifier, a sequence number, an event type, an optional product identifier, a referrer and a device.

3.3 Event types, referrers and devices

The HBase scan in Figure 4 shows the event types that occur in a session: home, search, category_listing, product_detail, cart and checkout. Product identifiers appear only on product_detail events. Two referrers (affiliate and search_engine) and two devices (tablet and desktop) are visible in that scan; the funnel analysis covers five referrers: email, affiliate, social, direct and search_engine.

3.4 Data preparation rules

- Timestamps are parsed from ISO-8601 strings in Spark before any time-based grouping.
- Invalid transaction rows are filtered before revenue is computed.
- Embedded line items are exploded into a normalised line-item table (one row per product in an order).
- Revenue uses embedded line-item subtotals *before* any order-level discount, because the source does not allocate discounts to individual products.
- Product pairs are formed only from distinct products in the same order, with canonical ordering (left < right) so each pair is counted once.

3.5 Data quality observations

The product subcategory field is documented inconsistently and the supplied generator omits it, so it is nullable in the product model and is not used in any result. The sample contains 2,000 transactions but only 1,001 sessions are linked to a purchase. This report does not establish why the two numbers differ (for example, whether some transactions refer to sessions outside the sample, whether sessions can contain several orders, or whether some transactions are filtered as invalid), so it is flagged as a point to verify in future runs rather than explained here.

4. Data modelling and technology selection

4.1 MongoDB

Four collections are used. *categories* stores each category with its embedded subcategory list; *products* has _id = product_id, name, category_id, inventory fields, and the embedded price_history array. The optional product subcategory field remains nullable because the supplied generator omits it. *users* has _id = user_id, nested geographic data, registration timestamp, and last-active timestamp. *transactions* has _id = transaction_id, user/session IDs, order totals/status, and an embedded items array. Embedding items avoids a join for order detail, which is normally read as a single business object.

Why embedding for orders and price history

An order and its line items have the same lifecycle: they are created together, read together, and rarely updated independently. Embedding therefore gives one document read per order and keeps the order consistent in a single atomic write. The same argument applies to price history inside a product and to subcategories inside a category. The trade-off is document growth, which is bounded here because an order has a limited number of items and a product has a short price history.

Indexes

Collection	Index	Purpose
products	(category_id, is_active)	List active products in a category
users	(geo_data.country)	Group or filter users by country
transactions	(timestamp desc)	Most recent orders; time-window scans
transactions	(user_id, timestamp desc)	A customer's order history, newest first
transactions	(items.product_id) multikey	Find orders containing a given product; supports unwinding items

The first aggregation unwinds items, joins product category, and calculates category revenue and units. The second implements recency/frequency/monetary segmentation from transactions. The exact code is in `mongodb/aggregations.py`.

MongoDB evidence

MongoDB Compass connected to `localhost:27017` shows the *ecommerce* database with its four collections (categories, products, transactions, users) (Figure 2 and Figure 3). The categories collection holds 25 documents, each with an embedded subcategories array, and the products collection holds 300 documents with the embedded price_history array, matching the sample sizes in Section 3. **MongoDB aggregation results are still pending:** the screenshots confirm the loaded data and schema only, so no aggregation counts or RFM segment results are claimed here. Run the commands in README.md and replace this note with the aggregation output.

Figure 2. MongoDB Compass: ecommerce.categories (25 documents, embedded subcategories array)

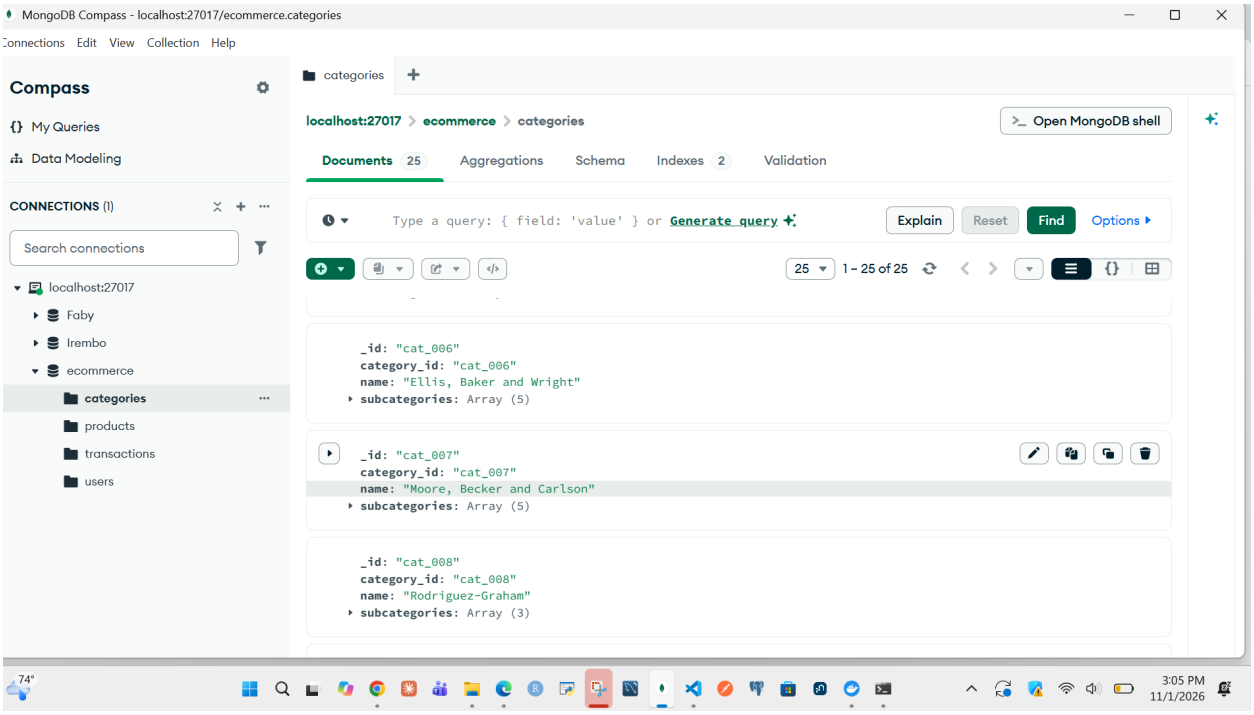
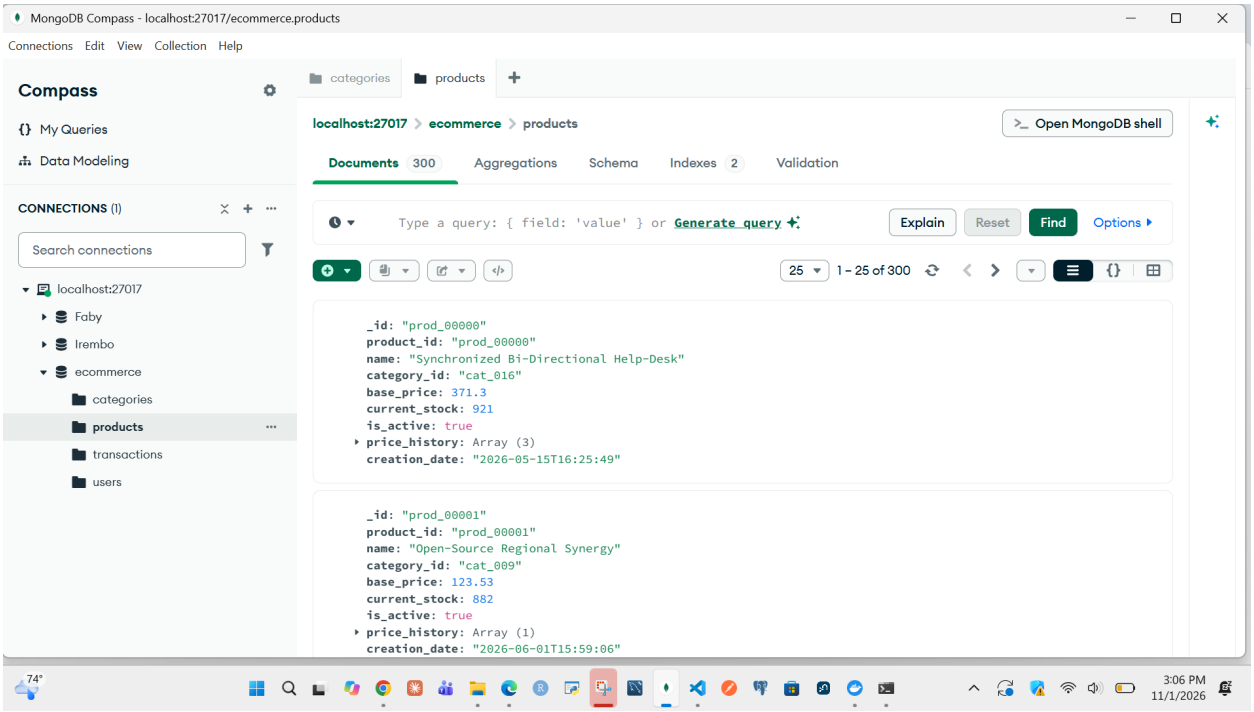


Figure 3. MongoDB Compass: ecommerce.products (300 documents, embedded price_history array)



4.2 HBase

user_events has row key `user_id|ISO-8601 timestamp|session_id|sequence` and two column families: *event* (type, session ID, optional product ID) and *context* (referrer and device). A prefix range scan therefore retrieves one user's chronological activity without scanning other users. Columns are sparse: non-product pages simply have no product qualifier.

product_daily_metrics has key `product_id|date` and metrics:views (with additional counters such as cart additions/purchases readily added). This table supports efficient one-product time-range scans. The design duplicates a small derived metric intentionally: HBase is query-first and does not support joins. The schema and loader are `hbase/schema.hbase` and `hbase/load_sessions.py`.

Table	Row key	Column families	Typical read
user_events	user_id|timestamp|session_id|sequence	event (type, session_id, product_id); context (referrer, device)	All events for one user in time order (prefix scan)
product_daily_metrics	product_id|date	metrics (views; other counters can be added)	One product over a date range

Row-key reasoning

HBase stores rows sorted by key, so the key is the only index. Putting the user identifier first groups a user's events together, and putting the ISO-8601 timestamp second makes them chronological because ISO-8601 strings sort lexicographically in time order. The session identifier and a zero-padded sequence number keep keys unique and preserve event order inside a session. The cost of this design is that queries by session or by referrer alone would need a full scan, which is exactly why those analyses are done in Spark rather than directly in HBase. In production, if a few users generated disproportionate write traffic, salting or pre-splitting would be needed to avoid hot regions.

HBase evidence

The HBase Master UI shows a running Apache HBase 3.0.0 deployment (Hadoop 3.4.3, ZooKeeper client 3.8.6, Eclipse Adoptium JVM 17.0.20.1) with one region server in state Normal and three regions (Figure 5 and Figure 6). The tables view (Figure 7) lists both designed tables, *product_daily_metrics* (family *metrics*) and *user_events* (families *context* and *event*), each ENABLED with one OPEN region and Snappy compression; the third region is presumably a system table. Figure 4 shows a prefix scan for `user_000042` run through `hbase/get_user_sessions.py` against 127.0.0.1:9090: the rows come back in chronological order and span two sessions, one from an affiliate referrer on a tablet (14 events, ending in cart and checkout) and one from a search-engine referrer on a desktop. Product IDs appear only on product_detail events, which illustrates the sparse columns.

Figure 4. HBase prefix scan of user_events for user_000042 (rows ordered by timestamp, session and sequence)

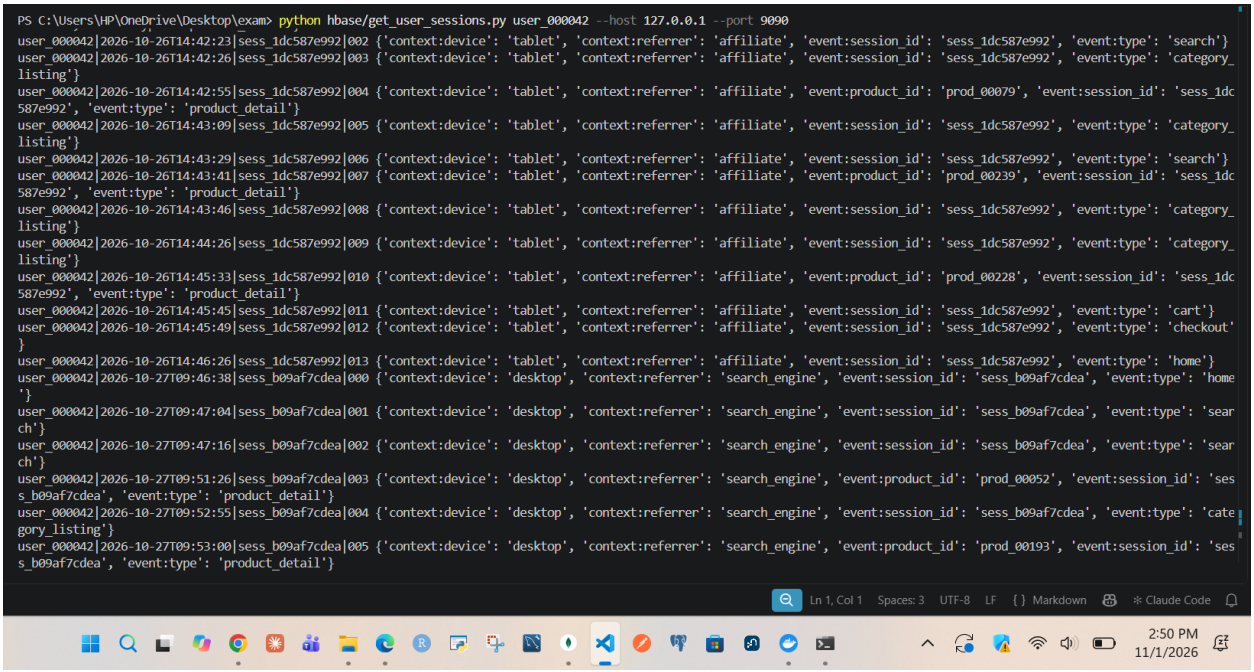


Figure 5. HBase Master UI: region server status and region count

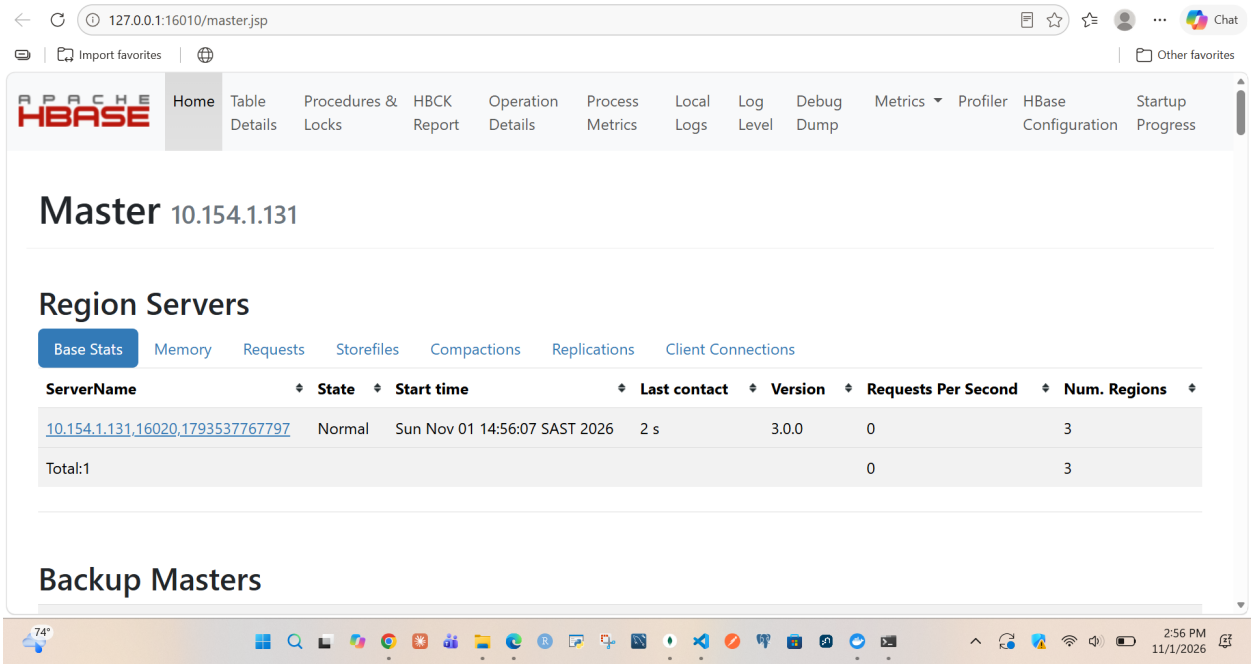
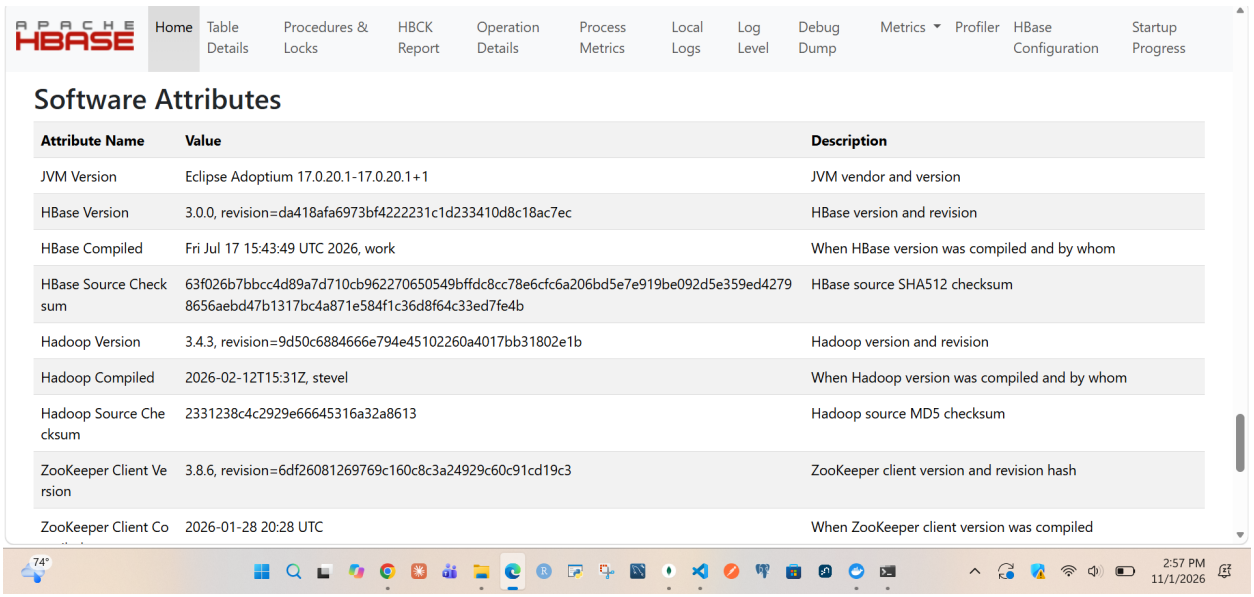


Figure 6. HBase Master UI: software attributes (HBase 3.0.0, Hadoop 3.4.3, ZooKeeper client 3.8.6)

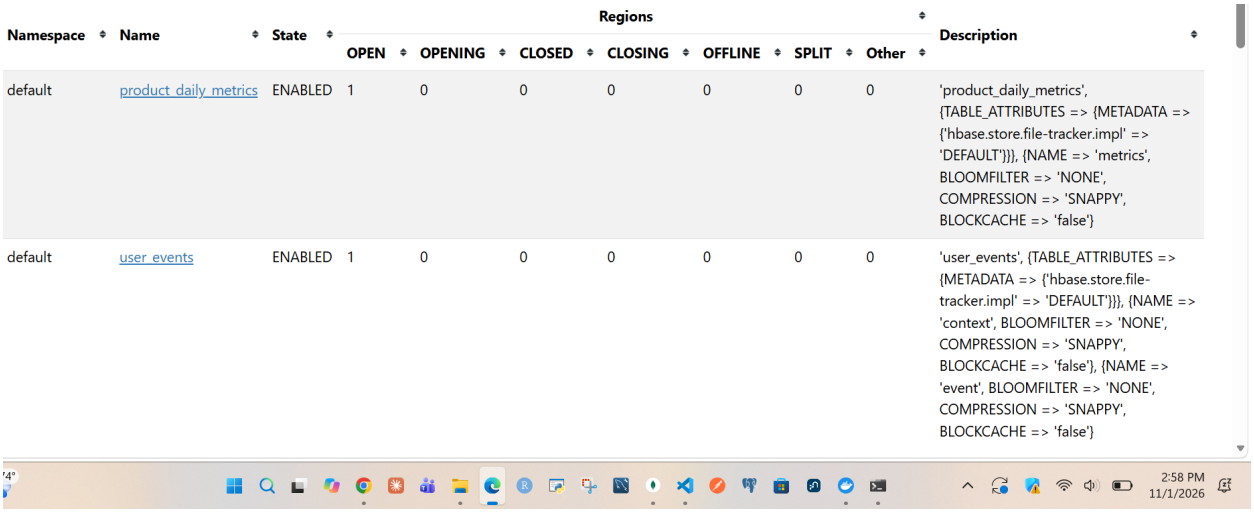


APACHE HBASE | Home | Table Details | Procedures & Locks | HBCK Report | Operation Details | Process Metrics | Local Logs | Log Level | Debug Dump | Metrics | Profiler | HBase Configuration | Startup Progress

Software Attributes

Attribute Name	Value	Description
JVM Version	Eclipse Adoptium 17.0.20.1-17.0.20.1+1	JVM vendor and version
HBase Version	3.0.0, revision=da418afa6973bf4222231c1d233410d8c18ac7ec	HBase version and revision
HBase Compiled	Fri Jul 17 15:43:49 UTC 2026, work	When HBase version was compiled and by whom
HBase Source Checksum	63f026b7bbcc4d89a7d710cb962270650549bffdc8cc78e6cfc6a206bd5e7e919be092d5e359ed42798656aebd47b1317bc4a871e584f1c36d8f64c33ed7fe4b	HBase source SHA512 checksum
Hadoop Version	3.4.3, revision=9d50c6884666e794e45102260a4017bb31802e1b	Hadoop version and revision
Hadoop Compiled	2026-02-12T15:31Z, stevel	When Hadoop version was compiled and by whom
Hadoop Source Checksum	2331238c4c2929e66645316a32a8613	Hadoop source MD5 checksum
ZooKeeper Client Version	3.8.6, revision=6df26081269769c160c8c3a24929c60c91cd19c3	ZooKeeper client version and revision hash
ZooKeeper Client Co	2026-01-28 20:28 UTC	When ZooKeeper client version was compiled

Figure 7. HBase Tables view: product_daily_metrics and user_events, both ENABLED with one OPEN region



Namespace	Name	State	Regions							Description
			OPEN	OPENING	CLOSED	CLOSING	OFFLINE	SPLIT	Other	
default	product_daily_metrics	ENABLED	1	0	0	0	0	0	0	'product_daily_metrics', {TABLE_ATTRIBUTES => {METADATA => {'hbase.store.file-tracker.impl' => 'DEFAULT'}}}, {NAME => 'metrics', BLOOMFILTER => 'NONE', COMPRESSION => 'SNAPPY', BLOCKCACHE => 'false'}
default	user_events	ENABLED	1	0	0	0	0	0	0	'user_events', {TABLE_ATTRIBUTES => {METADATA => {'hbase.store.file-tracker.impl' => 'DEFAULT'}}}, {NAME => 'context', BLOOMFILTER => 'NONE', COMPRESSION => 'SNAPPY', BLOCKCACHE => 'false'}, {NAME => 'event', BLOOMFILTER => 'NONE', COMPRESSION => 'SNAPPY', BLOCKCACHE => 'false'}

4.3 Choosing between MongoDB and HBase

Criterion	MongoDB	HBase
Data model	Rich nested documents	Sparse wide rows in column families
Best read pattern	Fetch an order, product or user as one object; flexible aggregation	Range scan by row-key prefix (one user, one product over time)
Joins	Lookup stage in aggregation (limited)	None; duplicate small derived data instead
Secondary access	Secondary and multikey indexes	Row key only (unless extra tables are maintained)
Write pattern	General purpose	Very high-volume append-oriented writes
Use in this project	Categories, products, users, transactions	User events, product-day metrics

MongoDB is superior for nested catalogue and transaction documents plus flexible aggregation. HBase is superior for many sparse events and predictable time-range access. The duplication is a deliberate trade-off for serving latency.

5. Spark batch processing and SQL

5.1 Pipeline overview

`spark/analytics.py` reads all JSON partitions, parses timestamps, filters invalid transaction rows, and explodes each embedded item into a normalized line item table. This makes revenue, basket pairs, and cohort calculations distributable. Revenue uses the embedded line-item subtotals before any order-level discount, because the source does not allocate discounts to individual products.

Step	Operation	Output
1	Read all JSON partitions; parse timestamps; filter invalid transaction rows	Clean users, products, transactions, sessions
2	Explode embedded items into one row per line item	Normalised line-item table
3	Join line items to products; group by category	Category revenue and units
4	Self-join line items within an order, left < right	Product-pair co-occurrence counts
5	Join purchase lines to users; group by registration and purchase month	Cohort revenue matrix
6	Spark SQL: join sessions to transactions on session_id; group by referrer	Funnel by referrer
7	Write compact aggregates to CSV from the driver	CSV results for charts and dashboard

5.2 Category revenue

Each exploded line item carries a product identifier and a line subtotal. Joining to the small product table adds the category identifier, and a group-by sums revenue and counts units. Because the catalogue has only 300 products, it is a natural candidate for a broadcast join in a larger deployment.

5.3 Product pairs (basket analysis)

Recommendation candidates are pairs of distinct products from the same order; canonical ordering (left < right) prevents double counting. For an order with *k* distinct products this produces k(k-1)/2 pairs, so the intermediate data grows quadratically with basket size. The score used is raw co-occurrence (the number of orders containing both products). This is the simplest form of association analysis (Agrawal et al., 1993) and does not yet correct for how popular each product is individually; support, confidence and lift are proposed as next steps in Section 9.

5.4 Cohort revenue

The cohort analysis joins purchase lines to users, groups customers by registration month, and sums revenue by later purchase month. The result is a matrix with registration month on one axis and purchase month on the other; each cell is the revenue earned in that purchase month from customers who registered in that registration month.

5.5 Funnel query in Spark SQL

Spark SQL implements the funnel query: session records represent HBase browsing data and transaction records represent MongoDB purchases. It groups by referrer and calculates session, engaged-session, converted-session, linked-purchase counts, and conversion rate. A session is counted as converted when its session identifier is joined to a qualifying transaction record.

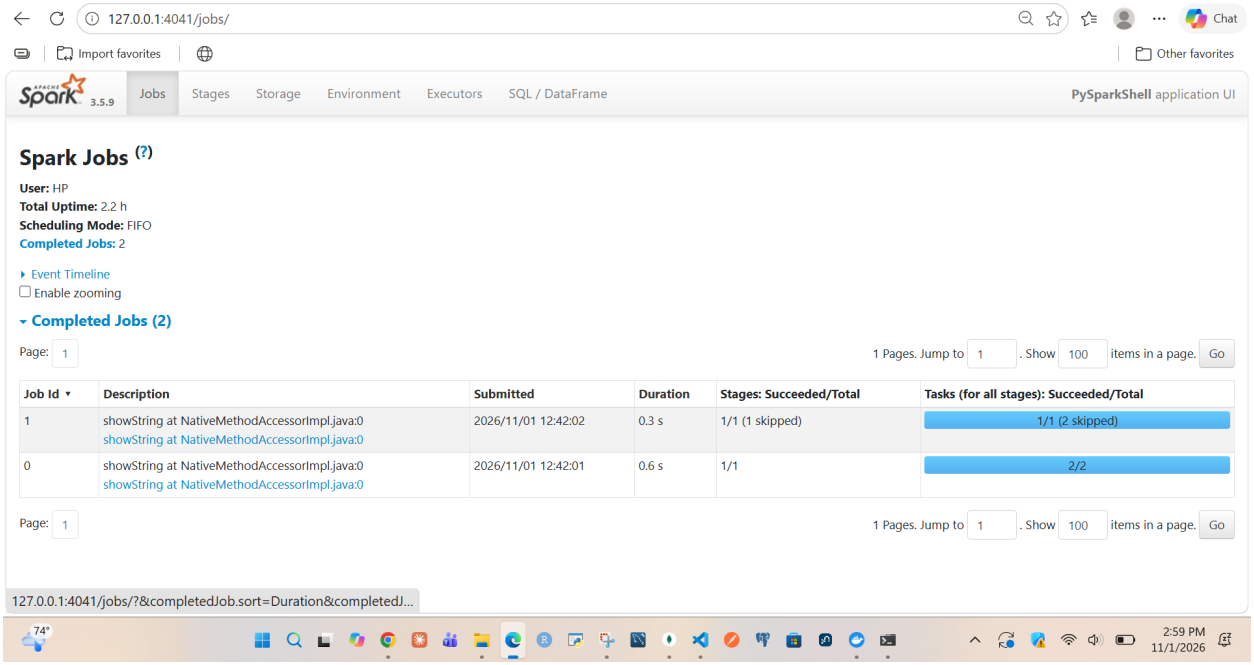
5.6 Scale-oriented implementation choices

For scale, the implementation reads session files from an explicit file list and selects only required columns before joins. It streams compact aggregate tables to CSV from the driver to avoid Hadoop's native Windows file-output dependency; on a cluster, use Spark's distributed CSV writer instead. Production runs should partition event data by date, broadcast the small product catalogue when appropriate, persist reused normalized tables, and avoid collecting large frames on the driver.

5.7 Spark evidence

The Spark 3.5.9 application UI (PySparkShell) shows two completed jobs, both `showString` actions with all stages and tasks succeeded and sub-second durations (Figure 8). Job 1 skipped a stage that Job 0 had already computed, which shows Spark reusing shuffle output between actions. The application had been up for about 2.2 hours and used FIFO scheduling.

Figure 8. Spark Jobs UI: two completed jobs, all tasks succeeded



6. Integrated analysis: acquisition-to-purchase funnel

Business question: Which acquisition channel produces the greatest share of converting sessions, and how should marketing budget be prioritized?

HBase supplies page-view/session engagement and referrer data. MongoDB supplies the linked purchase records. Spark joins session_id, aggregates by referrer, and produces a conversion percentage. This cannot be efficiently answered by performing a cross-system nested loop; Spark moves the computation to a parallel join and returns a compact result table. The funnel_by_referrer output and Figure 15 are the evidence for the decision.

6.1 Why a cross-system join is needed

Neither store can answer the question alone. HBase knows which referrer a session came from but not whether it ended in a purchase; MongoDB knows the purchases and their session identifiers but not the referrer. A nested-loop approach would fetch each session from HBase and look up its transaction in MongoDB, costing one round trip per session. Spark instead reads both sides in bulk and joins them on session_id in parallel.

6.2 Measured finding

Measured finding (10,000-session sample): email had the highest conversion rate at 10.76% (219 of 2,036 sessions). Search engine traffic had the lowest rate at 9.15% (182 of 1,990 sessions). Email also had the most sessions in this sample, so it led on both volume and conversion rate. The rates are close (a 1.61 percentage point gap), so the sample supports monitoring email performance rather than making a major budget shift without cost and significance data.

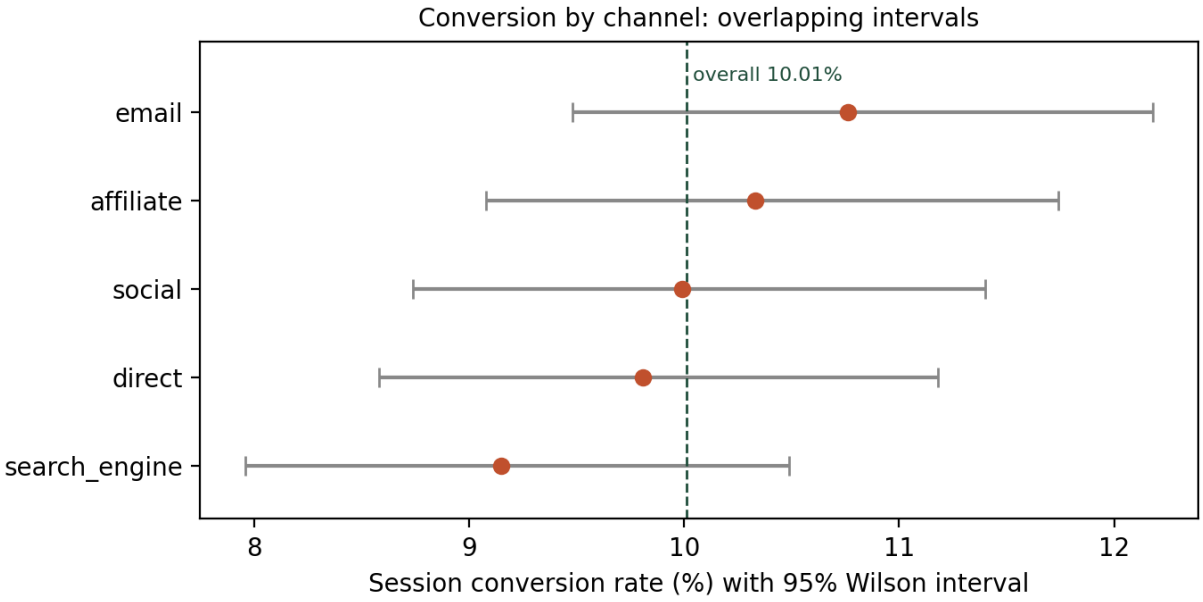
6.3 Statistical check added to this report

The original finding noted that significance data was missing. Using only the session and conversion counts above, two standard checks were computed. A chi-square test of independence across the five channels gives a statistic of 3.24 on 4 degrees of freedom ($p = 0.52$), so there is no evidence that conversion differs between channels in this sample. A two-proportion z-test of email against search engine gives $z = 1.71$ (two-sided $p = 0.088$), which does not reach the conventional 5% level. 95% Wilson intervals for every channel overlap heavily (Figure 9). These tests assume that sessions are independent and are applied to synthetic data, so they should be read as a guide to how much weight the gap deserves, not as a final verdict.

Channel	Sessions	Converted	Rate	95% interval
email	2,036	219	10.76%	9.48% to 12.18%
affiliate	2,013	208	10.33%	9.08% to 11.74%
social	1,952	195	9.99%	8.74% to 11.40%
direct	2,009	197	9.81%	8.58% to 11.18%
search_engine	1,990	182	9.15%	7.96% to 10.49%
All channels	10,000	1,001	10.01%	

Table: funnel by referrer. Intervals are Wilson 95% intervals computed for this report from the counts shown.

Figure 9. Conversion rate by channel with 95% Wilson intervals



6.4 Decision implications

- Email is a sensible channel to keep investing in, because it is at or near the top on both volume and rate.
- Search-engine traffic has the lowest observed rate, but the data do not show it is truly worse; a larger sample or a controlled test is needed before reducing spend.
- A budget decision needs acquisition cost per channel, which this synthetic data set does not contain.

7. Results and visualisations

The following figures were generated from the Spark outputs for the 10,000-session sample. They show descriptive results from synthetic data.

- cat_019 leads category revenue at 126,677.59 across 559 units, followed by cat_013 at 112,609.70 and cat_007 at 103,383.65.
- Product-pair frequency gives candidates for frequently bought together. The top pair appears in three orders, so evidence is sparse at this sample size.
- Email leads channel conversion at 10.76%; search engine traffic is lowest at 9.15%. Channel counts are around 2,000 sessions each.
- Cohort revenue spans 27 cohort/purchase-month combinations. The short synthetic activity window limits conclusions about long-term retention.

7.1 Category performance (Q1)

Gross line revenue across all categories is $1,895,377. The ten highest-revenue categories together account for $993,947, or 52.4% of that total, from 25 categories in all; if revenue were spread equally the top ten would hold 40%. The leader, cat_019, holds 6.7% of revenue against an equal share of 4%. Revenue is therefore moderately concentrated but not dominated by any one category: the gap between first and tenth place is $41,230.

Rank	Category	Revenue	Share of total
1	cat_019	$126,678	6.7%
2	cat_013	$112,610	5.9%
3	cat_007	$103,384	5.5%
4	cat_024	$100,005	5.3%
5	cat_018	$96,699	5.1%
6	cat_015	$94,648	5.0%
7	cat_001	$94,306	5.0%
8	cat_010	$93,447	4.9%
9	cat_002	$86,722	4.6%
10	cat_014	$85,448	4.5%

Table: top ten categories by revenue (line-item subtotals before order discounts). Shares are computed against $1,895,377.

Figure 10. Category revenue

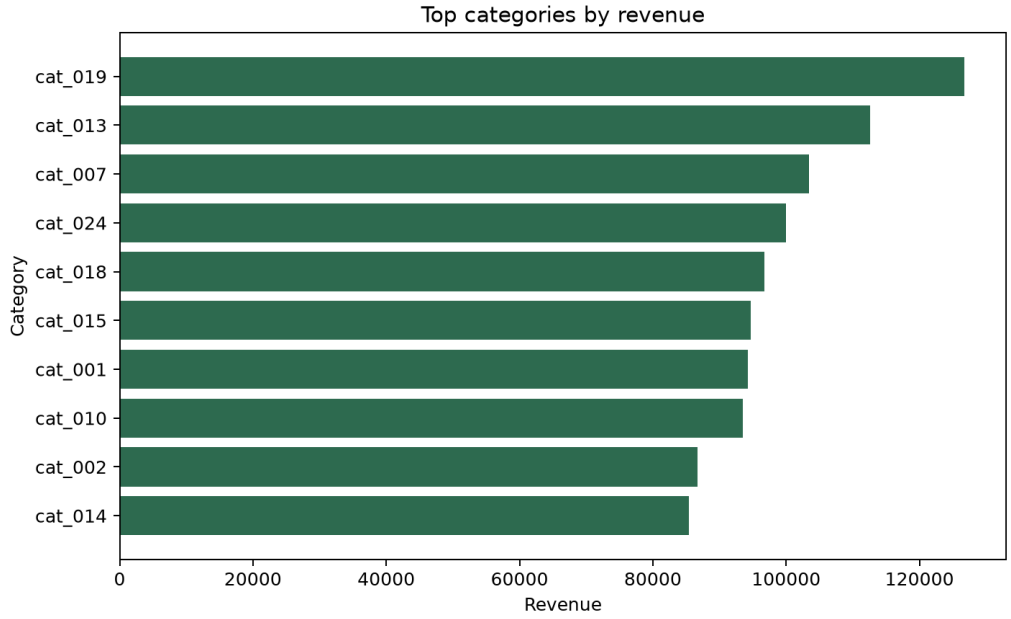
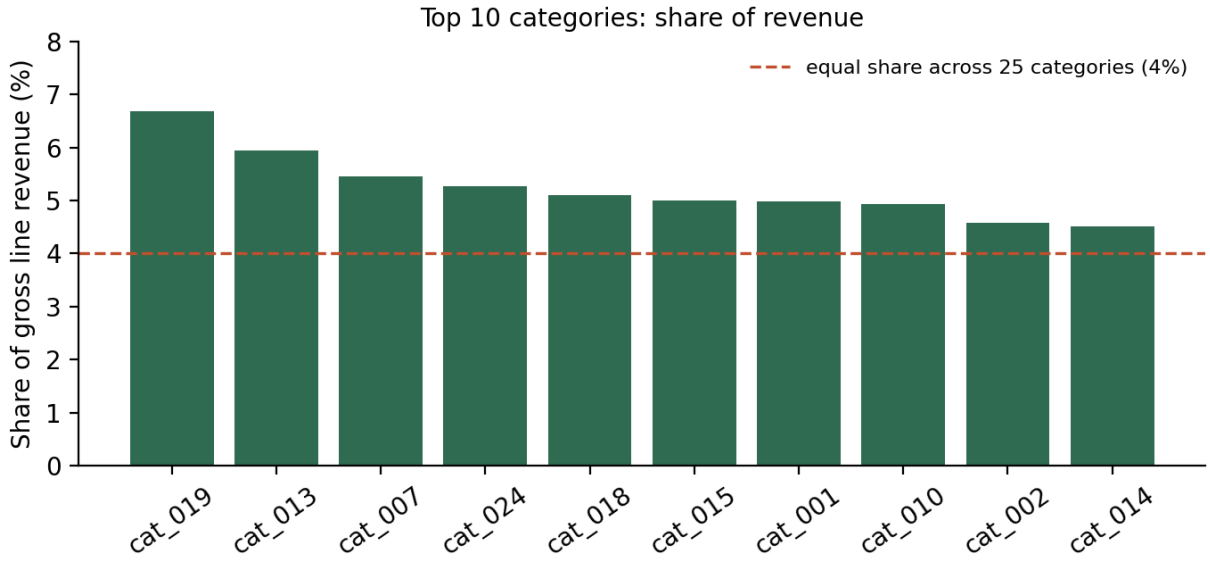


Figure 11. Share of gross line revenue held by each of the top ten categories



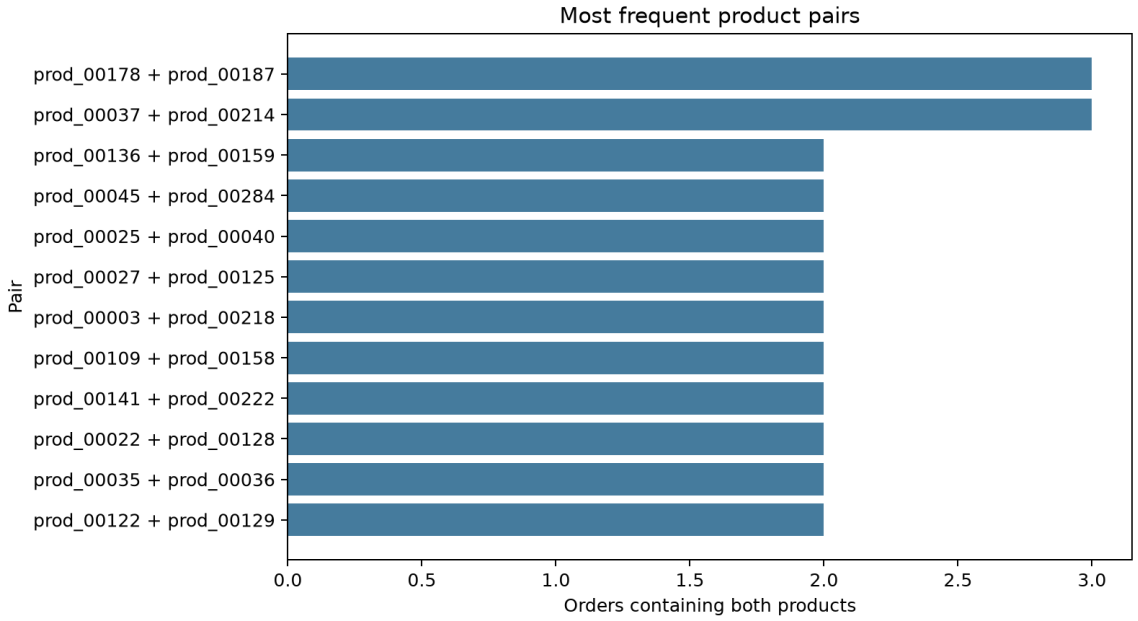
7.2 Frequently bought together (Q2)

Spark produced 2,682 distinct product pairs. Co-occurrence counts are very low: the two strongest pairs, prod_00037 with prod_00214 and prod_00178 with prod_00187, each appear in only three orders, and most listed pairs appear in two. With 2,000 orders and 300 products, almost every pair is seen at most once, so the ranking is dominated by chance. These pairs are candidates to examine, not reliable recommendations.

Product pair	Orders
prod_00037 + prod_00214	3
prod_00178 + prod_00187	3
prod_00001 + prod_00268	2
prod_00003 + prod_00044	2
prod_00003 + prod_00218	2
prod_00003 + prod_00299	2
prod_00004 + prod_00170	2
prod_00004 + prod_00296	2
prod_00005 + prod_00078	2
prod_00005 + prod_00115	2

Table: top pairs shown by the dashboard (ties at two orders are listed in the dashboard's own order).

Figure 12. Frequently purchased product pairs



7.3 Cohort revenue (Q3)

Cohorts are customers grouped by registration month (February to August 2026). Purchases appear in four months, August to November 2026. Summing every cell of the cohort matrix gives $1,895,377, which equals the gross line revenue on the dashboard, so the cohort table accounts for all revenue. Revenue by purchase month was $579,180 in August, $632,134 in September and $667,051 in October, rising each month; November contains only $17,012 because it is a partial month at the end of the window.

By registration cohort, the May cohort is the largest ($379,135), followed by April ($371,965) and March ($349,932). The August cohort is the smallest ($33,030) because those customers had at

most three months in the window, and the February cohort ($189,627) is much smaller than March to May, which may reflect the generator's registration distribution rather than customer behaviour. Within each cohort, October is usually the strongest month; the May cohort peaks at $154,310. Because there are no purchases before August, the matrix does not show whether customers buy soon after registering or later, so conclusions about retention are not supported.

Registered	2026-08	2026-09	2026-10	2026-11	Total
2026-02	$60,250	$64,981	$62,206	$2,190	$189,627
2026-03	$107,974	$115,783	$125,553	$622	$349,932
2026-04	$116,592	$115,522	$131,766	$8,085	$371,965
2026-05	$113,382	$109,768	$154,310	$1,675	$379,135
2026-06	$92,057	$102,886	$93,039	$1,637	$289,619
2026-07	$78,721	$109,986	$90,559	$2,803	$282,069
2026-08	$10,204	$13,208	$9,618	-	$33,030
Total	$579,180	$632,134	$667,051	$17,012	$1,895,377

Table: cohort revenue by registration month and purchase month (cell values from the dashboard; totals computed for this report).

Figure 13. Revenue by registration and purchase cohort

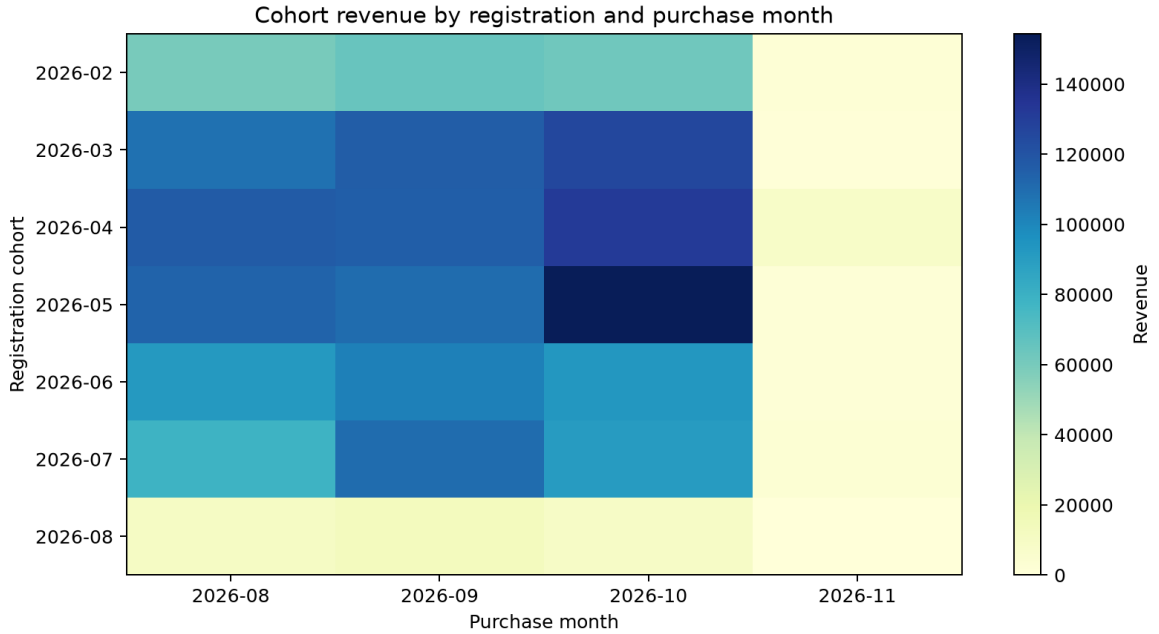
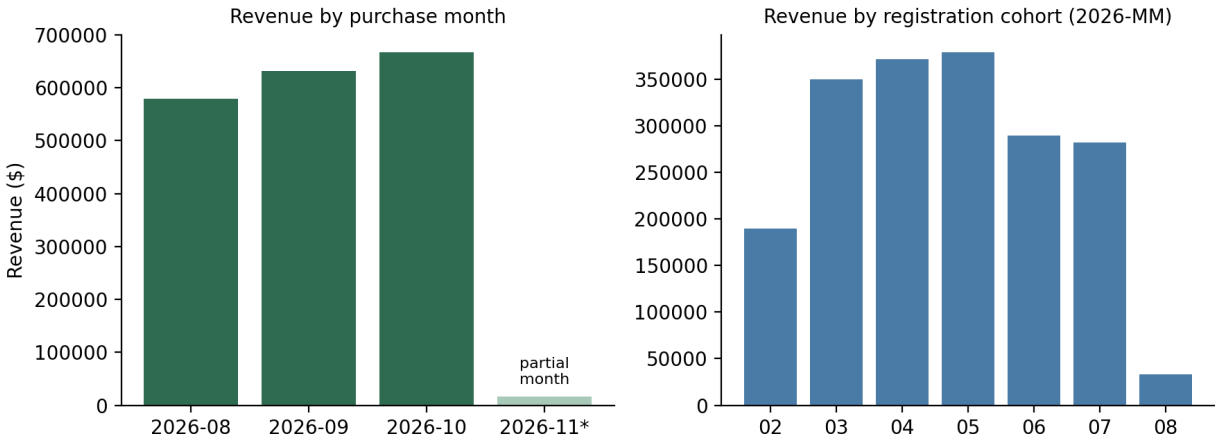


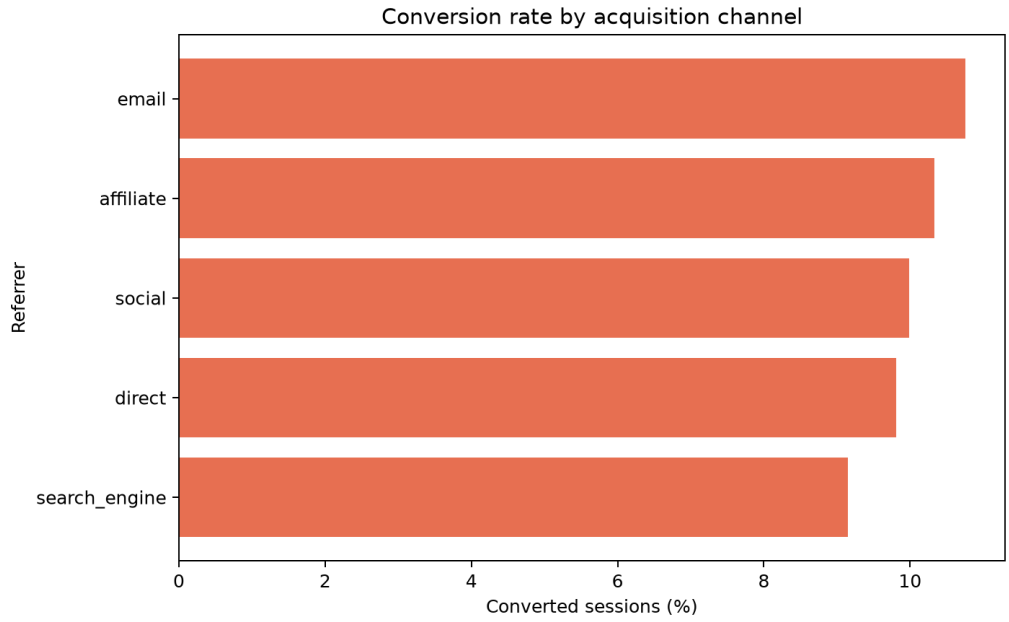
Figure 14. Revenue by purchase month and by registration cohort (totals computed from the cohort matrix; November is partial)



7.4 Acquisition funnel (Q4)

The funnel results are reported in Section 6. The conversion figure below is the original chart from the Spark outputs.

Figure 15. Conversion by referrer



7.5 Interactive dashboard

The same Spark outputs are served in an interactive dashboard. Figure 16 shows the headline metrics (10,000 sessions, 1,001 purchase-linked sessions, a 10.01% session conversion rate, and $1,895,377 gross line revenue), the category performance ranking, and the acquisition funnel table, which agree with the category and conversion figures above. Figure 17 shows the frequently bought together table and the cohort revenue heatmap with exact values per cell, which agree with the pair and cohort figures. The dashboard footer repeats that results are descriptive and come from a generated sample, not production customer or marketing data.

Figure 16. Dashboard: headline metrics, category performance and acquisition funnel

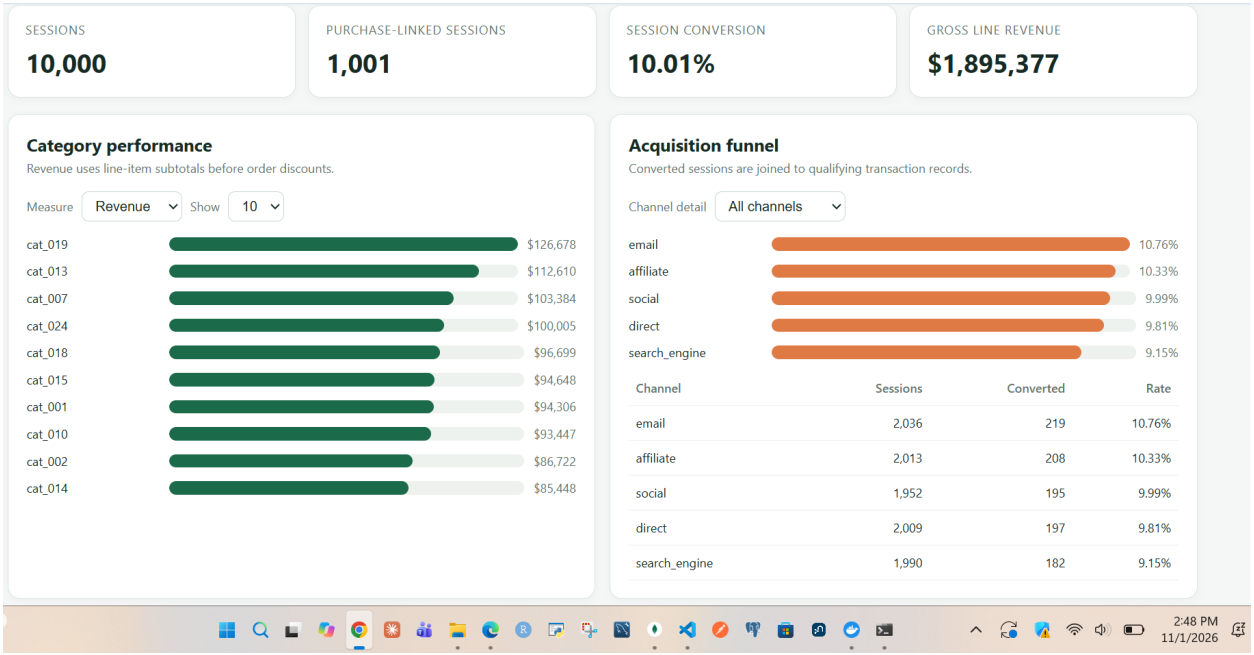
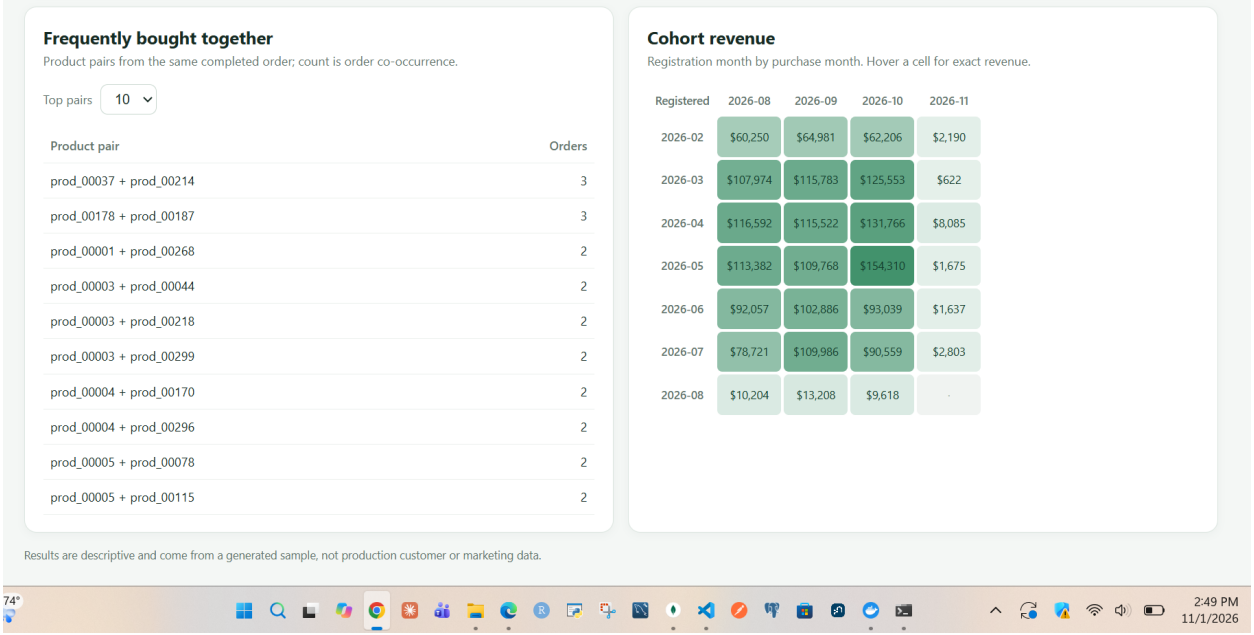


Figure 17. Dashboard: frequently bought together and cohort revenue



8. Discussion

8.1 Answers to the analytical questions

Question	Finding	Confidence
Q1 Category performance	cat_019 leads ($126,678, 6.7%); top ten categories hold 52.4% of revenue	Reasonable: based on all 2,000 transactions
Q2 Product affinity	Strongest pairs occur in only 3 orders	Low: counts are too small to separate signal from chance
Q3 Cohorts	Revenue grows each month Aug to Oct; May cohort largest	Moderate for totals; none for retention (short window)
Q4 Funnel	Email 10.76% highest, search engine 9.15% lowest	Low: differences not statistically significant (p = 0.52)

8.2 Business interpretation

For assortment, revenue is spread across many categories, so merchandising effort should not focus on a single category; the leaders are worth protecting in stock and promotion but the long tail still contributes nearly half of revenue. For cross-selling, the data are too sparse to power a recommender, but the pipeline for pair counting is in place and will become informative with more orders. For retention, the cohort matrix is the right structure, but a window that starts after most customers registered cannot show repeat behaviour. For marketing, email should be kept and monitored, and any change in spend should wait for cost data and a larger or controlled sample.

8.3 Validity and threats

- **Synthetic data.** Generated data has no real marketing cost, seasonality, or customer behaviour, so the numeric results illustrate the method and should not be used as business facts.
- **Sample size.** About 2,000 sessions per channel give 95% intervals about 2.5 to 3 percentage points wide, wider than the observed gaps between channels.
- **Multiple comparisons.** Choosing the best and worst of five channels after the fact exaggerates the gap; the chi-square test across all channels guards against that.
- **Revenue definition.** Line-item subtotals ignore order-level discounts, so true net revenue is lower than reported if discounts were applied.
- **Session-to-transaction link.** Only 1,001 sessions are linked to purchases although there are 2,000 transactions; the reason has not been established (Section 3.5).
- **Partial period.** November contains only a few days, so month-over-month comparisons that include it are misleading.

8.4 Performance and scalability

On a single machine the whole pipeline runs in seconds to minutes, as the sub-second Spark jobs in Figure 8 show. At production scale the main risks are different for each stage. The pair self-join grows quadratically with basket size, so very large baskets should be capped or sampled. The cross-system funnel join is best done by exporting events from HBase in bulk (for example with a snapshot or a scan with a column filter) rather than by many random reads. HBase writes should be pre-split and salted if user activity is skewed. Event data should be partitioned by date so that daily jobs read only new partitions, and tables reused across analyses should be persisted.

8.5 Design trade-offs revisited

Storing browsing events in HBase and orders in MongoDB means two systems to run, secure and keep consistent, and the identifier that links them (session_id) must be generated and stored identically in both. The benefit is that each system is used for what it does best: one-object reads and flexible aggregation for orders, and cheap range scans over sparse events. Duplicating product-day metrics adds a write path that must be kept in step with the raw events, in return for fast single-product time-range reads.

9. Limitations and future work

9.1 Limitations

The source is synthetic, so it lacks realistic marketing cost, product text, and customer demographics needed to measure profitability or causal lift. Product subcategory is inconsistently documented and is nullable in this implementation. The present recommendation score is co-occurrence, not personalized ranking. MongoDB aggregation output has not yet been captured, so the category revenue and RFM segmentation implemented in MongoDB are not evidenced here; the category figures in this report come from the Spark pipeline.

9.2 Future work

- **Association metrics.** Add support, confidence and lift for product pairs so popular products do not dominate, or use Spark ALS with explicit interaction data for personalised ranking.
- **Run the MongoDB aggregations** and add their output, then compare MongoDB and Spark category revenue as a cross-check.
- **Longer, more realistic data.** Generate activity that starts at registration so cohort retention can be measured, and add acquisition cost per channel.
- **Controlled test.** Validate the email advantage with an experiment or a much larger sample.
- **Investigate the session link.** Explain why 1,001 sessions are linked to 2,000 transactions.
- **Production hardening of HBase.** Pre-splitting or salting if a few users became write hotspots, retention policies for raw events, and authenticated connectors.
- **Cluster execution.** Use Spark's distributed CSV writer, date partitioning, broadcast joins and persisted intermediate tables.

10. Conclusion

The project built and evidenced a polyglot analytics pipeline: MongoDB for nested business documents, HBase for sparse time-ordered events, and Spark for the joins and aggregations that cross both. The pipeline produced category revenue, product-pair, cohort and funnel results from a 10,000-session synthetic sample, and the Spark jobs, HBase deployment, HBase prefix scan and dashboard are supported by screenshots.

The substantive findings are modest and are reported with their limits. Revenue is moderately concentrated, with cat_019 leading at 6.7% of the total. Product-pair counts are too low to support recommendations. Cohort revenue rises from August to October, but the window is too short to measure retention. Email has the highest observed conversion at 10.76%, but a chi-square test ($p = 0.52$) and a two-proportion test ($p = 0.088$) show that the channel differences could easily be chance. The most important outcome is therefore the working, documented pipeline and the discipline of reporting only what has been executed: the MongoDB aggregation results remain the one analysis step still to be evidenced.

11. Reproducibility

Commands and prerequisites are in `README.md`. The Spark run used Spark 3.5.9, Java 17, and the sample sizes listed in Section 3. The HBase deployment (version 3.0.0) and a user_events prefix scan are documented in Figure 4 to Figure 7, the Spark session in Figure 8, and the loaded MongoDB collections in Figure 2 and Figure 3. MongoDB aggregation output remains to be captured; do not report that section as executed until evidence is added.

11.1 Suggested run order

- Start MongoDB and HBase (and the HBase gateway on port 9090).
- Run the JSON generator to create users, products, transactions and sessions.
- Load users, products and transactions into MongoDB; load sessions and product-day metrics into HBase.
- Run the MongoDB aggregations and save their output (pending).
- Run `spark/analytics.py` to produce the CSV result tables.
- Generate the static charts and open the dashboard from the CSV results.
- Run `hbase/get_user_sessions.py` for a sample user to demonstrate the prefix scan.

References

Agrawal, R., Imielinski, T. and Swami, A. (1993). Mining association rules between sets of items in large databases. Proceedings of the ACM SIGMOD International Conference on Management of Data, pp. 207-216.

Apache Software Foundation. Apache HBase Reference Guide. https://hbase.apache.org/book.html

Apache Software Foundation. Apache Spark Documentation, version 3.5. https://spark.apache.org/docs/3.5.0/

Chang, F., Dean, J., Ghemawat, S., et al. (2006). Bigtable: A distributed storage system for structured data. Proceedings of the 7th USENIX Symposium on Operating Systems Design and Implementation (OSDI).

Dean, J. and Ghemawat, S. (2004). MapReduce: Simplified data processing on large clusters. Proceedings of the 6th USENIX Symposium on Operating Systems Design and Implementation (OSDI).

MongoDB, Inc. MongoDB Manual: Data modeling, Indexes and Aggregation. https://www.mongodb.com/docs/manual/

Wilson, E. B. (1927). Probable inference, the law of succession, and statistical inference. Journal of the American Statistical Association, 22(158), 209-212.

Zaharia, M., Xin, R. S., Wendell, P., et al. (2016). Apache Spark: A unified engine for big data processing. Communications of the ACM, 59(11), 56-65.

Appendix A. Data dictionary

Fields are those documented in the data model and visible in the screenshots. Fields marked "documented" are described in the model but their exact types are not shown in the screenshots.

categories (MongoDB, 25 documents)

Field	Type	Description
_id	string	Category identifier, e.g. cat_007
category_id	string	Same as _id
name	string	Category name
subcategories	array	Embedded subcategory list (3 to 5 items in the documents shown)

products (MongoDB, 300 documents)

Field	Type	Description
_id	string	Product identifier, e.g. prod_00000
product_id	string	Same as _id
name	string	Product name
category_id	string	Reference to categories
base_price	number	Base price (e.g. 371.3)
current_stock	integer	Inventory level
is_active	boolean	Whether the product is active
price_history	array	Embedded price changes (1 to 3 items in the documents shown)
creation_date	string	ISO-8601 timestamp
subcategory	string, nullable	Optional; omitted by the supplied generator

users (MongoDB, 500 documents; documented)

Field	Type	Description
_id	string	User identifier, e.g. user_000042
geo_data	object	Nested geographic data, including country
registration timestamp	string	When the user registered; defines the cohort month
last-active timestamp	string	Most recent activity

transactions (MongoDB, 2,000 documents; documented)

Field	Type	Description
_id	string	Transaction identifier
user_id	string	Reference to users
session_id	string	Link to the browsing session; used by the funnel join
timestamp	string	Order time
totals / status	number / string	Order totals and status; invalid rows are filtered in Spark
items	array	Embedded line items with product_id and line subtotal

user_events (HBase)

Part	Name	Description
Row key	user_id|timestamp|session_id|sequence	Unique and chronologically ordered per user
event	type	home, search, category_listing, product_detail, cart, checkout
event	session_id	Session identifier
event	product_id	Present only on product_detail events (sparse)
context	referrer	email, affiliate, social, direct or search_engine
context	device	e.g. tablet or desktop

product_daily_metrics (HBase)

Part	Name	Description
Row key	product_id|date	One row per product per day
metrics	views	Daily view count; further counters can be added

Appendix B. Evidence checklist

Component	Evidence in this report	Status
Data sample sizes	Section 3; dashboard headline counts	Evidenced
MongoDB collections loaded	Figure 2 and Figure 3	Evidenced (categories, products); users and transactions shown in sidebar only
MongoDB aggregations	None captured	**Pending**
HBase deployment	Figure 5, Figure 6, Figure 7	Evidenced
HBase prefix scan	Figure 4	Evidenced
Spark execution	Figure 8; CSV outputs and charts	Evidenced
Category, pair, cohort, funnel results	Figure 10 to Figure 13, Figure 16 to Figure 17	Evidenced
Statistical tests	Section 6.3 (computed from reported counts)	Computed for this report

Appendix C. Environment and versions

Item	Value	Source
HBase	3.0.0, compiled Fri Jul 17 2026	HBase Master UI
Hadoop	3.4.3	HBase Master UI
ZooKeeper client	3.8.6	HBase Master UI
JVM (HBase)	Eclipse Adoptium 17.0.20.1	HBase Master UI
Spark	3.5.9 (PySparkShell)	Spark UI
Java for Spark	17	Original report
MongoDB access	MongoDB Compass at localhost:27017, database ecommerce	Compass
HBase gateway	127.0.0.1, port 9090	Scan command
HBase region server	1 server, 3 regions, state Normal	HBase Master UI
Operating system	Windows (Windows-specific Hadoop output note)	Report and screenshots

Appendix D. Glossary

Term	Meaning
Polyglot persistence	Using several storage technologies, each for the workload it suits best
Embedded document	A sub-document or array stored inside its parent document, read in one operation
Multikey index	A MongoDB index over an array field, with an entry per array element
Row key	The sorted primary key of an HBase row; the only built-in index
Column family	A group of HBase columns stored together on disk
Prefix (range) scan	Reading all HBase rows whose key starts with a given prefix
Salting / pre-splitting	Techniques to spread HBase writes across regions and avoid hot spots
Explode	Spark operation turning each element of an array column into its own row
Broadcast join	Join in which a small table is copied to every worker to avoid a shuffle
Session conversion	Share of sessions that are linked to at least one qualifying purchase
Cohort	Group of customers sharing a registration month
Co-occurrence	Number of orders containing both products of a pair
Lift	Ratio of observed co-occurrence to that expected if the two products were independent
Wilson interval	A confidence interval for a proportion that behaves well for moderate sample sizes
Chi-square test	Test of whether observed counts differ from those expected under independence